Project Documentation: AI Plant Disease Detector

Date: October 26, 2025 **Goal:** To build a web application that can identify plant diseases from images of leaves using Artificial Intelligence (specifically, a Convolutional Neural Network or CNN).

Phase 1: Setting Up Your Computer (Initial Plan: Local Setup)

The first idea was to build and run everything directly on your ASUS Vivobook laptop.

1. **Create Project Folder:**
    - We created a main folder for the project on your Desktop called Plant_disease_project. This is where all the files would live.
2. **Identify Necessary Code Files:**
    - We determined we needed three main text files:
        - requirements.txt: Lists all the Python libraries the project needs (like TensorFlow, Streamlit, etc.).
        - train.py: The Python script that would teach (train) the AI model to recognize diseases using lots of images.
        - app.py: The Python script for the final web application that uses the trained model.
3. **Realize Model Files Were Missing:**
    - We realized that the actual "brain" of the AI (the trained model file, which ends up being called plant_disease_model.keras) and the list of disease names (class_names.txt) didn't exist yet. These files are *created* by running the train.py script, which we hadn't done.
4. **Troubleshooting - Step 1: Fixing pip**
    - When you tried to install libraries using the command pip install ..., you got the error: 'pip' is not recognized...
    - **Why?** Windows didn't know where the pip program was. This usually means Python wasn't added to the system's PATH (a list of places Windows looks for programs) during installation.
    - **Fix 1 (Try py -m pip):** We tried using py -m pip install .... This tells Windows to use the Python launcher (py) to find and run pip. This also failed initially.
    - **Fix 2 (Reinstall Python & Add to PATH):** The most reliable fix was to reinstall Python.

        - You downloaded the Python installer.
        - Crucially, during installation, you needed to check the box **"Add Python to PATH"**.
    - **Fix 3 (Manual PATH Edit):** When pip and py still weren't found even after reinstalling, we had to manually tell Windows where Python was.
        - We found your Python installation folder (e.g., C:\Users\deven\AppData\Local\Programs\Python\Python311).
        - We added *two* paths (the main Python folder AND its Scripts subfolder) to your Windows "Environment Variables" under the "Path" setting.
        - After closing and reopening the command prompt, pip and py finally worked!

5. **Troubleshooting - Step 2: Fixing TensorFlow Installation**
    - When you ran py -m pip install tensorflow ..., you got the error: ERROR: No matching distribution found for tensorflow.
    - **Why (First Guess - 32-bit Python):** This often means you have a 32-bit version of Python, but modern TensorFlow needs 64-bit Python. We checked using platform.architecture() and found you correctly had **64-bit** Python.
    - **Why (Real Reason - Python Too New):** We then checked your Python version (py --version) and found it was 3.14.0. This version was too new! TensorFlow hadn't released a compatible version for it yet.
    - **The Fix:**
        - We uninstalled Python 3.14.
        - We downloaded and installed **Python 3.11 (64-bit)**, which is known to be compatible. Again, checking **"Add Python to PATH"** was essential during installation.
    - **Success:** After installing Python 3.11, the command py -m pip install tensorflow ... finally worked, along with installing opencv-python-headless, numpy, matplotlib, scikit-learn, kaggle, streamlit, and Pillow.

Phase 2: Getting Data & Preparing to Train (Local Setup)

Now that the libraries were installed, we needed the image data to train the model.

1. **Getting the Kaggle API Key:**
    - To download the data using code, we needed an API key from Kaggle.

- We navigated Kaggle: Click profile icon (top right) -> "Account" (or "Settings") -> Scroll down to the **API** section.
- You clicked **"Create New API Token"**, which downloaded the kaggle.json file.

2. **Downloading the Dataset (using kaggle command):**
   - **Troubleshooting Command:** First, we tried py -m kaggle datasets download ... which failed because kaggle isn't run that way. The correct command was just kaggle datasets download ....
   - **Troubleshooting Authentication:** You then got a KeyError: 'username'. This meant the kaggle command couldn't find your API key.
   - **The Fix:** You moved the downloaded kaggle.json file into the correct folder: C:\Users\deven\.kaggle\kaggle.json.
   - **Troubleshooting Rules:** The command ran but didn't download, asking you to accept rules.
   - **The Fix:** You went to the dataset page on Kaggle (link) and clicked "Download" to accept the terms.
   - **Success:** Running kaggle datasets download ... again successfully downloaded plantvillage-dataset.zip to your project folder. (We also noted you accidentally started a browser download of archive.zip, which we cancelled/ignored).
3. **Unzipping the Dataset:**
   - You used Windows' built-in "Extract All..." feature on plantvillage-dataset.zip.
4. **Fixing the Folder Structure (The "Double Folder"):**
   - Unzipping created an extra folder layer. Your path was Plant_disease_project\plantvillage-dataset\plantvillage dataset\color, but the code needed Plant_disease_project\plantvillage dataset\color.
   - **The Fix:** You went inside the first plantvillage-dataset (hyphen) folder, **cut** the plantvillage dataset (space) folder, went back up to Plant_disease_project, and **pasted** it there. You then deleted the empty hyphenated folder and the zip file.
5. **Creating the Code Files:**
   - You realized the train.py, app.py, and requirements.txt files were missing from the project folder.
   - I provided the full Python code for train.py (the training script) and app.py (the Streamlit web app), and the text content for requirements.txt.

- **Saving Files Correctly:** You saved these files using Notepad.
- **Troubleshooting File Extensions:** When saving, Notepad added .txt to the end (e.g., train.py.txt). We saw this using the ls (or dir) command in the terminal.
- **The Fix:** We used the ren (rename) command in the terminal to fix the filenames (e.g., ren train.py.txt train.py).

Phase 3: Trying to Train Locally & GPU Problems

Everything was finally set up correctly on your laptop.

1. **Running the Training (py train.py):**
   - You started the training script.
   - **Problem:** It was extremely slow (estimating many hours).
2. **Identifying the GPU:**
   - I explained this slowness was because it was using your CPU (main processor).
   - You confirmed your laptop has an **NVIDIA RTX 3050** graphics card (GPU), which should make training much faster.
3. **Attempt 1: Install GPU TensorFlow (tensorflow[and-cuda]):**
   - We stopped the slow training (Ctrl+C).
   - Uninstalled the basic tensorflow.
   - Installed the special GPU package: py -m pip install "tensorflow[and-cuda]".
   - **Troubleshooting Dependencies:** This caused new errors (missing scipy, then incompatible numpy). We fixed these by installing scipy and then forcing a reinstall of everything: py -m pip install --upgrade --force-reinstall "tensorflow[and-cuda]" numpy opencv-python-headless scipy.
4. **Verifying GPU Detection:**
   - We ran the training again (py train.py), but it was still slow.
   - We ran a diagnostic command: py -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))".
   - **Result:** It printed [] (an empty list), meaning TensorFlow *could not see* your RTX 3050.
5. **Attempt 2: Update NVIDIA Drivers:**

- The most common reason for the GPU not being detected is outdated main graphics drivers.
- You opened **GeForce Experience**, checked for updates, and installed the latest driver.
- You **restarted** your laptop.
- We ran the diagnostic command again. **Result:** Still []. The GPU was not detected.

6. **Identifying the Real Problem (TF Version vs. Windows):**
   - After research, I found the core issue: **TensorFlow versions 2.11 and newer no longer support using GPUs directly on Windows** (they require a complex setup called WSL2). The tensorflow[and-cuda] package is primarily for Linux/WSL2.
   - Your installed TensorFlow (likely 2.13 or newer from the previous step) could not use your GPU on Windows.
7. **Attempt 3: Manual CUDA/cuDNN Installation (Complex):**
   - The "correct" but very difficult solution was to downgrade TensorFlow to version **2.10** (the last version with Windows GPU support) and manually install the matching **CUDA Toolkit (11.2)** and **cuDNN library (8.1)** from NVIDIA's website, including copying specific files into system folders.
   - You rightly pointed out this was too complicated and time-consuming.

Phase 4: Switching to Google Colab (The Fast & Easy Way)

We decided to abandon the difficult local GPU setup and use Google's free online service.

1. **What is Colab?** Explained it's a free online Jupyter Notebook environment run by Google that gives you access to powerful hardware, including GPUs, through your browser.
2. **Setting up Colab:**
   - Opened colab.research.google.com and created a new notebook.
   - Changed the runtime type to use a **"T4 GPU"** hardware accelerator.
   - Uploaded your kaggle.json API key using a code cell (from google.colab import files; files.upload()).
   - Moved the key to the correct location using !cp kaggle.json ~/.kaggle/.
3. **Getting Data in Colab:**

- Downloaded the dataset *directly to the Colab computer* using !kaggle datasets download .... This was much faster than your home Wi-Fi because it uses Google's fast internet.
- Unzipped the dataset using !unzip -q plantvillage-dataset.zip.
- **Troubleshooting File Not Found:** The unzip failed because your Colab session had likely timed out (Colab deletes files if idle), or you hadn't accepted the Kaggle rules.
- **The Fix:** We re-ran the download and unzip commands.
- **Troubleshooting Unzip Prompt:** The unzip asked if you wanted to replace files (because you ran it twice).
- **The Fix:** You correctly typed A (for All) and pressed Enter.

4. **Training on Colab:**
    - Pasted the entire train.py code (slightly adjusted for Colab paths) into a single code cell.
    - Ran the cell.
    - **Troubleshooting Slow Speed:** It was still taking ~10-13 minutes per epoch, slower than expected. We verified the runtime was set to T4 GPU. We ran the diagnostic tf.config.list_physical_devices('GPU') which printed [ ].
    - **The Fix:** We did a **"Factory reset runtime"** to get a completely fresh environment. We re-uploaded the Kaggle key, re-downloaded, and re-unzipped the data.
    - **Success:** The diagnostic command now printed Num GPUs Available: 1. Training started again.
    - **Accepting Speed:** The training now took ~9-10 minutes per epoch. While not the absolute fastest possible, this was confirmed to be using the GPU and much better than the local CPU. You let the training complete (10 epochs).
5. **Downloading the Trained Model:**
    - After training finished, Colab automatically created plant_disease_model.keras and class_names.txt.
    - You used the file browser panel on the left side of Colab, right-clicked on these two files, and selected "Download" to save them to your laptop.

Phase 5: Running the App Locally (Using the Colab Model)

Now we used the model trained on Colab with the app.py script on your local laptop.

1. **Setup:** You moved the downloaded plant_disease_model.keras and class_names.txt into your local Plant_disease_project folder, alongside app.py and requirements.txt.
2. **Running the App:** You ran streamlit run app.py in your local terminal.
3. **Troubleshooting Model Loading - Error 1 (batch_shape):**
    - The Streamlit app showed an error: Unrecognized keyword arguments: ['batch_shape'].
    - **Why?** The TensorFlow version used in Colab (we checked, it was 2.19.0) was different from the version on your laptop (likely 2.13.1 or 2.10.x from earlier attempts).
    - **The Fix:** You installed the matching version locally: py -m pip install tensorflow==2.19.0.
4. **Troubleshooting Model Loading - Error 2 (expects 1 input(s), received 2):**
    - After fixing the version, a new error appeared: Layer "dense_1" expects 1 input(s), but it received 2 input tensors.
    - **Why?** This often indicates a subtle incompatibility or corruption in the saved .keras file format when moving between environments.
    - **Attempted Fix 1 (Re-download):** You re-downloaded the .keras file, but the error persisted.
    - **Attempted Fix 2 (Use .h5 format):**
        - In Colab, you added code to load the .keras model and re-save it in the older .h5 format (model.save('plant_disease_model.h5')).
        - You downloaded plant_disease_model.h5.
        - You updated app.py locally to load .h5 instead of .keras.
        - **Result:** The error *still* occurred.
5. **Troubleshooting Model Loading - Error 3 (File not found):**
    - While trying the .h5 fix, you briefly got a File not found error for .keras.
    - **Why?** We looked at your files using dir. Your downloaded .keras file was named plant_disease_model (1).keras.
    - **The Fix:** You renamed it correctly using ren "plant_disease_model (1).keras" plant_disease_model.keras.
6. **Troubleshooting Model Loading - Final Fix (Load Weights Separately):**

- Since loading the full model file kept failing, we used the most robust method.
- **In Colab:** You ran code to load the model and save *only its weights*: model.save_weights('plant_disease_model_weights.weights.h5'). You downloaded this .weights.h5 file.
- **Locally:**
  - You deleted the full model file (.keras or .h5).
  - You placed the .weights.h5 file in your project folder.
  - You significantly **updated app.py**. The load_model_and_classes function was rewritten to:
    1. First, build the *exact same model structure* (MobileNetV2 base + pooling + dropout + dense layer) using TensorFlow code.
    2. Then, load *only the learned weights* from the .weights.h5 file into that structure using model.load_weights(...).
- **SUCCESS!** Running streamlit run app.py with this updated code finally worked. The model loaded correctly.

7. **Final App Touches:**
   - **Fixing Warning:** The app showed a yellow warning: use_column_width parameter has been deprecated... use use_container_width.
   - **The Fix:** You edited app.py and changed st.image(..., use_column_width=True) to st.image(..., use_container_width=True).
   - **Testing:** You uploaded an image. The app correctly identified that the model's confidence (45.40%) was below the 50% threshold and displayed the warning message we programmed earlier, proving that feature worked.
8. **Understanding the Result:**
   - Explained that the app running in your browser at http://localhost:8501 is a **local web application** created by Streamlit.
   - Explained the **"Deploy"** button is an option to publish your app to Streamlit Community Cloud (requires GitHub), making it a public website accessible to anyone.

Conclusion

You successfully built an AI Plant Disease Detector!

- You set up your local Python environment.
- You used Google Colab to train a deep learning model (MobileNetV2) on the PlantVillage dataset using a powerful GPU.
- You downloaded the trained model weights and class names.
- You built a local web application using Streamlit (app.py) that loads the model, takes an image upload, preprocesses the image, makes a prediction, and displays the result with confidence and treatment advice.
- You implemented important features like a confidence threshold to handle uncertain predictions.
- You overcame significant technical challenges related to software versions, dependencies, file paths, and model loading!

Your next step, if you choose, is to deploy this application online using the "Deploy" button and GitHub. Congratulations on completing a complex end-to-end AI project!